Looking at the path to search for the word “`antic`”, all the four nodes shown and traversed share the same prefix, “`ant`”: wouldn’t it be nice if we could somehow compress these nodes, and only store the common prefix once, and just the deltas for each node?

6.1.3 Description and API

The data structure that we are going to introduce in the next section was created to answer these needs, and also to offer an efficient way to solve another operation: find all the keys in the container that starts with the same prefix.

From our example in the previous sub-section, in fact, you can already see that if we were, somehow, able to store the common prefixes of strings only once, we might as well be able to quickly access all strings starting with those prefixes.

Table 6.1 shows the public API for an abstract data structure (*ADT*) that supports the usual containers basic operations, plus two new ones: retrieving all the strings starting with a certain prefix, and finding the longest prefix of a string stored into the container.

From what we have seen in the previous example, `PrefixTree` could be a good name as well for these data structure, although string container is at the same more generic (as an abstract data structure, we don’t care if it’s implemented using a tree or some other concrete counterpart), and it conveys the gist of this container: being specific for strings. The fact that prefix search is supported is